

Stack - Linked List Implementation

Last Updated : 13 Sep, 2025

A stack is a linear data structure that follows the **Last-In-First-Out (LIFO)** principle. It can be implemented using a [linked list](#), where each element of the stack is represented as a node. The head of the linked list acts as the top of the stack.

Declaration of Stack using Linked List

A stack can be implemented using a **linked list** where we maintain:

- A Node structure/class that contains:
 - data → to store the element.
 - next → pointer/reference to the next node in the stack.
- A pointer/reference top that always points to the current **top node** of the stack.

Initially, top = null to represent an empty stack.

[C++](#)[Java](#)[Python](#)[C#](#)[JavaScript](#)

```
/* Node structure */
class Node {
    public int data;
    public Node next;

    public Node(int x) {
        data = x;
        next = null;
    }
}

/* Stack class */
class MyStack {

    // pointer to top node
    private Node top;

    public MyStack() {
        // initially stack is empty
    }
}
```



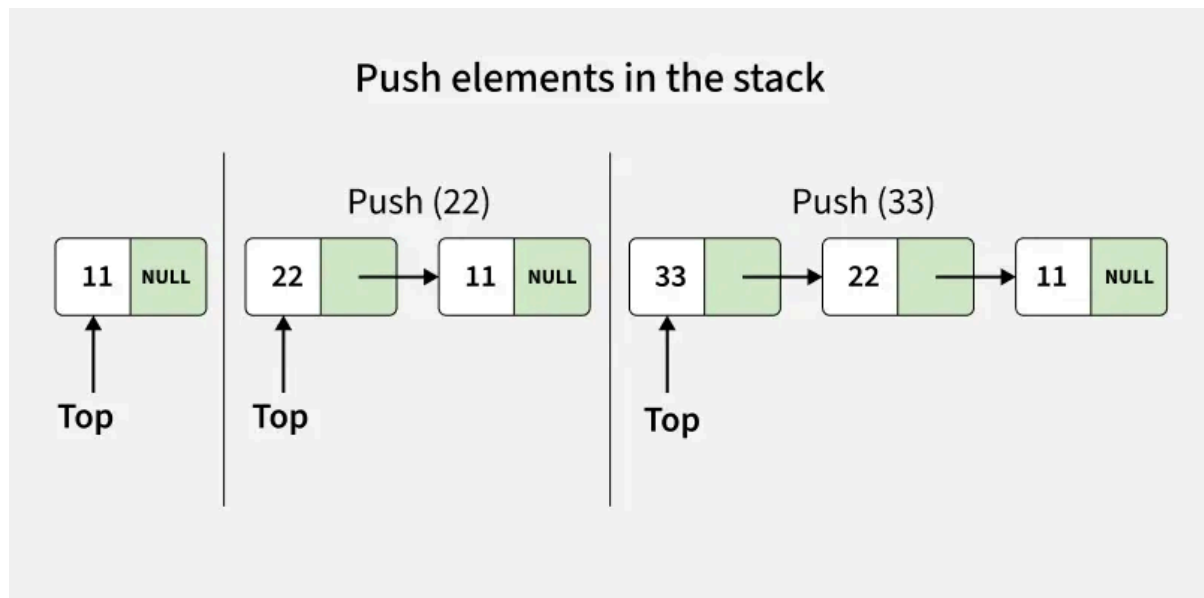
```
        top = null;  
    }  
}
```

Operations on Stack using Linked List

Push Operation

Adds an item to the stack. Unlike array implementation, there is no fixed capacity in linked list. Overflow occurs only when memory is exhausted.

- A new node is created with the given value.
- The new node's next pointer is set to the current top.
- The top pointer is updated to point to this new node.

**C++****Java****Python****C#****JavaScript**

```
void push(int x) {  
    Node temp = new Node(x);  
    temp.next = top;  
    top = temp;  
}
```



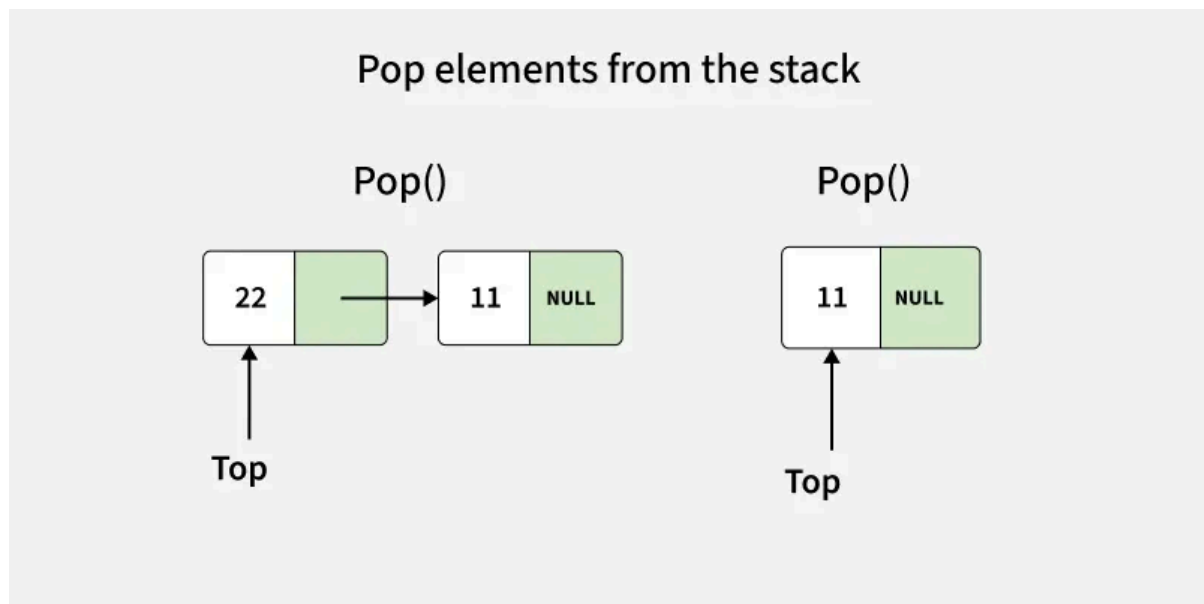
Time Complexity: $O(1)$

Auxiliary Space: $O(1)$

Pop Operation

Removes the top item from the stack. If the stack is empty, it is said to be an **Underflow condition**.

- Before deleting, we check if the stack is empty (`top == NULL`).
- If the stack is empty, underflow occurs and deletion is not possible.
- Otherwise, we store the current top node in a temporary pointer.
- Move the top pointer to the next node.
- Delete the temporary node to free memory.

**C++****Java**

Python

C#

JavaScript

```
public int pop() {  
    if (top == null) {  
        System.out.println("Stack Underflow");  
        return -1;  
    }  
  
    Node temp = top;  
    top = top.next;  
    int val = temp.data;  
  
    temp = null;  
    return val;  
}
```

Time Complexity: $O(1)$

Auxiliary Space: $O(1)$

Peek (or Top) Operation

Returns the value of the top item without removing it from the stack.

- If the stack is empty (`top == NULL`), then no element exists.
- Otherwise, simply return the data of the node pointed by `top`.

C++**Java**

Python

C#

JavaScript

```
int peek() {  
  
    if (top == null) {  
        System.out.println("Stack is Empty");  
        return -1;  
    }  
  
    return top.data;  
}
```

Time Complexity: $O(1)$

Auxiliary Space: $O(1)$

isEmpty Operation

Checks whether the stack has no elements.

- If the top pointer is `NULL`, it means the stack is empty and the function returns `true`.
- Otherwise, it returns `false`.

C++**Java**

Python

C#

JavaScript

```
boolean isEmpty() {  
    return top == null;  
}
```

Time Complexity: $O(1)$

Auxiliary Space: $O(1)$

Stack Implementation using Linked List

C++**Java**

Python

C#

JavaScript

```
// Node structure  
class Node {  
    int data;
```

```
Node next;

Node(int x) {
    data = x;
    next = null;
}
}

// Stack implementation using linked list
class myStack {
    Node top;

    // To Store current size of stack
    int count;

    myStack() {
        // initially stack is empty
        top = null;
        count = 0;
    }

    // push operation
    void push(int x) {
        Node temp = new Node(x);
        temp.next = top;
        top = temp;

        count++;
    }

    // pop operation
    int pop() {
        if (top == null) {
            System.out.println("Stack Underflow");
            return -1;
        }
        Node temp = top;
        top = top.next;
        int val = temp.data;

        count--;
        return val;
    }
}
```

```
// peek operation
int peek() {
    if (top == null) {
        System.out.println("Stack is Empty");
        return -1;
    }
    return top.data;
}
```

Array Strings Linked List Stack Queue Tree Graph Searching

Sign In

```
// check if stack is empty
boolean isEmpty() {
    return top == null;
}

// size of stack
int size() {
    return count;
}

public static void main(String[] args) {
    myStack st = new myStack();

    // pushing elements
    st.push(1);
    st.push(2);
    st.push(3);
    st.push(4);

    // popping one element
    System.out.println("Popped: " + st.pop());

    // checking top element
    System.out.println("Top element: " + st.peek());

    // checking if stack is empty
    System.out.println("Is stack empty: " + (st.isEmpty()
? "Yes" : "No"));

    // checking current size
    System.out.println("Current size: " + st.size());
}
}
```

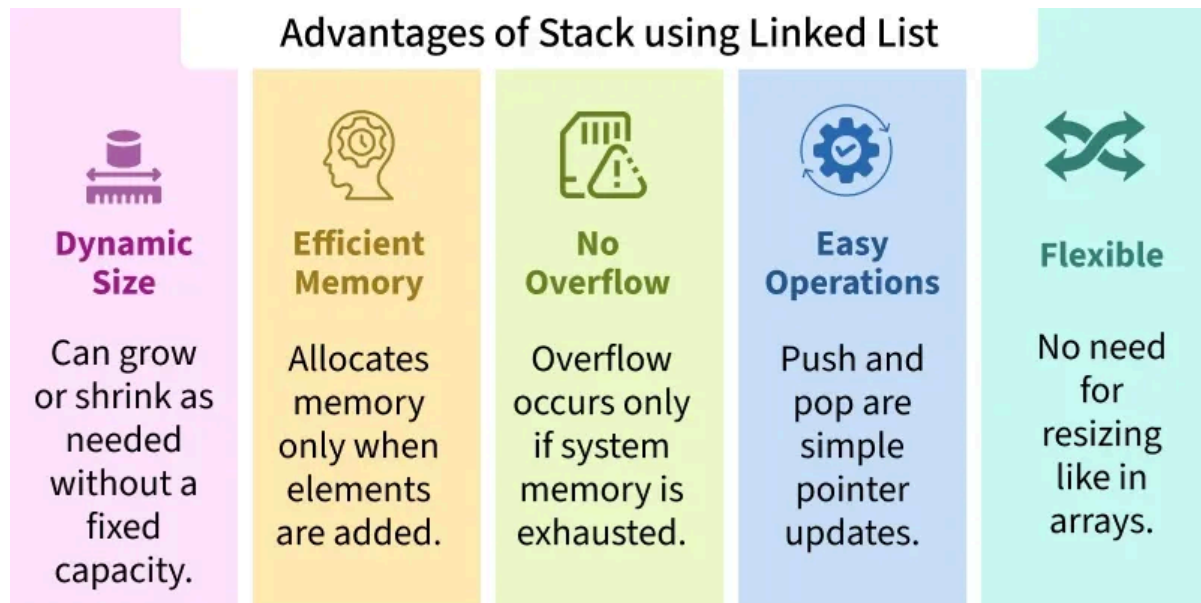
Output

Popped: 4

Top element: 3

Is stack empty: No

Current size: 3



Linked
Impler
of Stac



Linked
Impler
of Stac



Linked
Impler
of Stac
Pythor



Linked List Implementation
of Stack in C++

[Visit Course](#)[Comment](#)[More info](#)

Explore

[DSA Fundamentals](#)[Data Structures](#)[Algorithms](#)[Advanced](#)[Interview Preparation](#)[Practice Problem](#)

Corporate & Communications Address:

A-143, 7th Floor, Sovereign Corporate
Tower, Sector- 136, Noida, Uttar Pradesh
(201305)

Registered Address:

K 061, Tower K, Gulshan Vivante
Apartment, Sector 137, Noida, Gautam
Buddh Nagar, Uttar Pradesh, 201305



Company

[About Us](#)[Legal](#)[Privacy Policy](#)[Contact Us](#)[Advertise with us](#)

Explore

[POTD](#)[Job-A-Thon](#)[Community](#)[Blogs](#)[Nation Skill Up](#)

GFG Corporate Solution
Campus Training Program

Tutorials

Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core Subjects
Interview Preparation
GATE
Software and Tools

Videos

DSA
Python
Java
C++
Web Development
Data Science
CS Subjects

Courses

IBM Certification
DSA and Placements
Web Development
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Preparation Corner

Aptitude
Puzzles
GfG 160
DSA 360
System Design

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved